

CS429 Assignment 4

Roxanne Krause, Kurukulasuriya Leitan, Marnina Willard

April 24, 2026

1 Task 1

The high-level steps in data preparation are as follows.

1. Use Python library *yfinance* to download all data relating to a single company stocks for each day over a specified period, including
 - Closing price at end of day
 - High price of the day
 - Low price of the day
 - Opening price at the beginning of day
 - Volume traded
2. Clean the data.
3. Split the data into training and testing sequence windows containing all of the above features for a specific number of days, using the target of the day after sequence end's closing price for prediction; additionally, we standardize the data in this section.
4. Convert the window sequences and labels into *PyTorch* tensors for use in training.

Unless otherwise stated in other sections, the default targets and parameters used in data collection are as follows.

- **Companies:** Microsoft, Amazon, NVIDIA, Google
- **Data Start Date:** January 1, 2020
- **Data End Date:** January 1, 2024
- **Training Set Percentage:** 80%
- **Testing Set Percentage:** 20%
- **Window Size:** 60
- **Scaler:** Min Max Scaling

The coming sections provide more details of the techniques used to prepare this dataset, focusing on cleaning and splitting.

1.1 Cleaning

The following are the specific cleaning steps used when preparing this data set for model use.

1. Remove all *yfinance* artifacts.
2. Remove all rows with missing, or null, values.

yfinance leaves a row specifying the company the column data belongs to as a way of easing use when pulling information for multiple companies at once. We chose to keep all company information completely separate to ensure no accidental leakage in implementation, so this row is completely stripped from the returned data, instead organizing into separate CSV files for persistence.

Further, all rows with missing values are removed, deciding to not use back or forward filling to guess the values. This is achieved simply with the Python library *pandas*.

1.2 Splitting

The following are the steps taken to split, standardize, and prepare the data for model use. This is accomplished in a somewhat unusual way, thus this section is provided to explain the splitting mechanism used.

Before showing the technique, let us clarify some definitions of wording used below:

- Let p be the true portion of training data desired.
- Let *numRawTrain* be the number of raw data samples for creating training sequences.
- Let *numRawTest* be the number of raw data samples for creating testing sequences.
- Let *totalRawSamples* be the total number of raw data samples.
- Let *windowSize* be the number of samples included in a sequence window.
- Let *numSamplesInSet* refer contextually to the number of samples in a given data set.

The process of splitting is conducted as follows.

1. Calculate *numRawTrain*, the number of raw data samples needed to have p portion of sequence windows (training set) come from the first cut of the raw data, and $1 - p$ portion (testing set) come from the second cut.
2. Split the raw data, allowing the raw sample interval $[0, \text{numRawTrain})$ to be allotted for creating training sequence windows.

3. Split the raw data, allowing the raw sample interval $[numRawTrain, totalRawSamples)$ to be allotted for creating testing sequence windows.
4. Fit the scaler to the raw training data set. Transform this data set with the fitted scaler.
5. Transform the raw testing data set with the fitted scaler.
6. For both raw training and testing sets, transform the sets into sliding window sequences, saving the next day's closing price as the target prediction.

A unique tactic was taken to make the training and testing set creation more malleable to raw data set and window size diversity in Step 1 above. The initial tactic we used was to simply split the raw data by portion p , and then continue to scaling. However, given a small enough raw data set (which was often achieved pulling a single year's worth of data) or varying window sizes, we encountered issues with only having a single test window to test accuracy on, or worse, none at all. This felt inappropriate, so a simple calculation is used to instead guarantee the p portion of total sequence windows, not p portion of raw data is used for training, and the rest for testing. This is done to ensure that the model is being properly tested on ample sequences.

In brief, we can derive a calculation from the way we create the windows. To avoid subtle data leakage between training and testing sets, we continue to split the raw data first. This way we can fit a scaler to a training set without any risk of having accidental influence from test data and avoid subtle leaks of testing into training from window overlap. Then, we utilize a simple sliding window technique on each of these disjoint, scaled sets, in which we create $numSamplesInSet - windowSize$ windows, each overlapping the prior except for a single sample.

From this technique, we know that the total number of windows created from these split sets of raw data can be expressed as:

$$\begin{aligned}
 & (numRawTrain - windowSize) + (numRawTest - windowSize) \\
 & = numRawTrain + numRawTest - 2 * windowSize \\
 & = totalRawSamples - 2 * windowSize
 \end{aligned}$$

Therefore, we know we want a goal of $p * (totalRawSamples - 2 * windowSize)$ windows in our training sequence set. We then need to solve for the number of raw data samples needed to achieve this number of windows. As we have seen, we can express the number of windows yielded by the raw training data as: $numRawTrain - windowSize$. Therefore, by setting our desired number of sequence windows in the training set equal to the number of training windows given an unknown $numRawTrain$, we can solve for this single unknown.

$$\begin{aligned}
 p * (totalRawSamples - 2 * windowSize) & = numRawTrain - windowSize \\
 numRawTrain & = p * (totalRawSamples - 2 * windowSize) + windowSize
 \end{aligned}$$

From this *numRawTrain* (which is rounded as needed using Python’s *round()*), we may split the raw data in such a way that not only flexibly and cleanly creates training and testing sets, but also ensures we are testing the model sufficiently.

Once this calculation is complete for Step 1 above, we continue on with the remaining steps as normal to properly scale the data and then split it for model use.

2 Task 2

The baseline model is a single-layer RNN implemented in *PyTorch* as the **StockRNN** class. The network takes as input a sequence of $M = 60$ days, where each day is represented by a 5-dimensional feature vector (closing price, daily high, daily low, opening price, and trading volume), and outputs a single scalar: the predicted closing price for the following day ($N = 1$).

2.1 Architecture

At each timestep $t \in \{1, \dots, M\}$, the recurrent layer updates its hidden state according to the standard RNN recurrence:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

where $x_t \in \mathbb{R}^5$ is the input feature vector at day t , $h_t \in \mathbb{R}^d$ is the hidden state of dimension d , and W_h , W_x , and b are the learned weight matrices and bias. The hidden state is initialized to zero at the start of each sequence. After processing all M timesteps, only the final hidden state h_M is passed to a fully connected linear layer to produce the output:

$$\hat{y} = W_o h_M + b_o$$

where $\hat{y} \in \mathbb{R}$ is the predicted next-day closing price.

2.2 Intuition

The intuition behind this design is that stock prices are inherently sequential, the context of past prices informs future movement. The recurrent layer maintains a hidden state that acts as a compressed memory of the sequence seen so far, allowing the model to capture temporal dependencies across the 60-day window. We use only the final hidden state for prediction since h_M already encodes the full history by recurrence. The subsequent linear layer then maps this representation to a single continuous value, making the network an end-to-end regression model.

3 Task 3

3.1 Hyperparameter Selection

The following hyperparameters were used to train the baseline RNN:

- **Hidden size:** 32 for Microsoft, Amazon, and Google; 64 for NVIDIA, which exhibits higher price volatility and benefits from a larger capacity model.
- **Epochs:** 100
- **Learning rate:** 0.001 (Adam optimizer)
- **Batch size:** 32 (mini-batch gradient descent)
- **Loss function:** Mean Squared Error (MSE)

Adam was selected over standard SGD for its adaptive learning rate, which improves convergence stability on time-series data. Mini-batches of size 32 provide a good balance between gradient noise and computational efficiency. The hidden size for NVIDIA was doubled after observing higher training error relative to the other stocks at the default size of 32.

3.2 Loss Curves

Figure 1 shows the MSE training loss over 100 epochs for each stock.

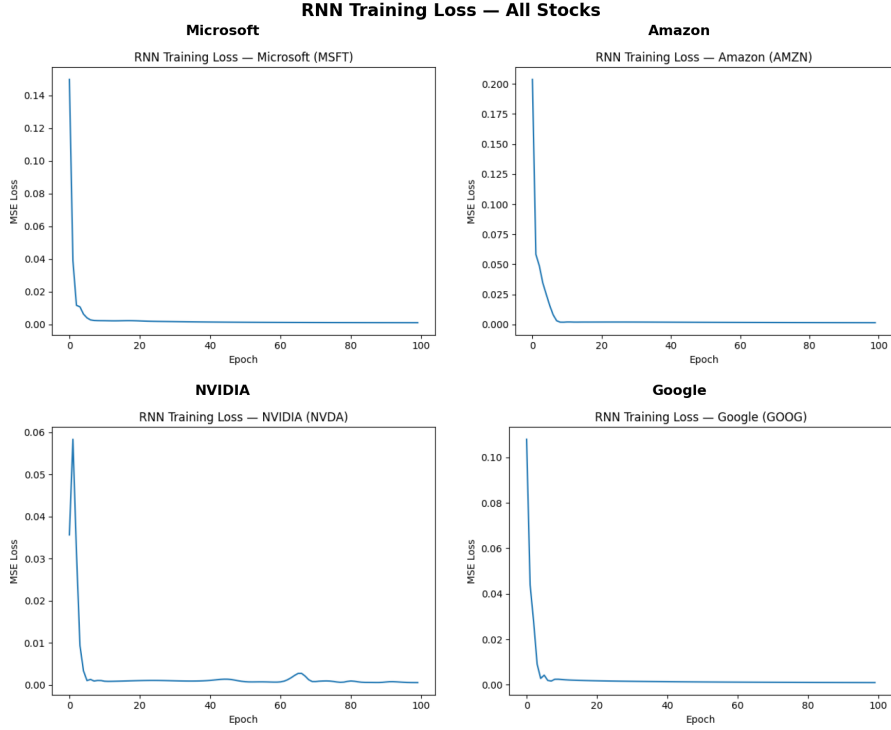


Figure 1: RNN training loss (MSE) over 100 epochs for all four stocks.

3.3 Results

Table 1 summarizes the training time, training error, and test error for each stock. Error is reported as Mean Absolute Percentage Error (MAPE), and accuracy as $100 - \text{MAPE}$.

Stock	Train Time (s)	Train MAPE (%)	Test MAPE (%)	Test Accuracy (%)
Microsoft	8.14	1.68	1.84	98.16
Amazon	8.22	2.38	1.98	98.02
NVIDIA	9.37	2.75	4.01	95.99
Google	8.14	1.70	1.45	98.55

Table 1: RNN baseline results per stock.

The model performs well across all four stocks, achieving test accuracies above 95%. NVIDIA is the most challenging stock, with the highest test MAPE of 4.01%, reflecting its greater price volatility during this period. The gap between training and test MAPE for NVIDIA (2.75% vs. 4.01%) indicates mild overfitting that the baseline RNN struggles to overcome. The remaining three

stocks show test MAPE lower than or comparable to their training MAPE, suggesting the model generalizes well without significant overfitting.

4 Task 4

The GRU model (**StockGRU**) extends the baseline by replacing the recurrent layer with two stacked Gated Recurrent Unit (GRU) layers. Unlike the baseline RNN, a GRU introduces two learned gates; a reset gate and an update gate that give the network fine-grained control over how much prior context to retain or discard at each timestep. This mechanism directly addresses the vanishing-gradient problem that limits the baseline RNN on longer sequences.

4.1 Architecture

At each timestep $t \in \{1, \dots, M\}$, the GRU computes:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (1)$$

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (2)$$

$$\tilde{h}_t = \tanh(W_n x_t + r_t \odot (U_n h_{t-1}) + b_n) \quad (3)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (4)$$

where r_t is the reset gate, z_t is the update gate, $\tilde{h}_t$ is the candidate hidden state, and $\odot$ denotes elementwise multiplication. When $z_t \approx 0$, the unit preserves the previous hidden state nearly unchanged; when $z_t \approx 1$, it replaces it with the new candidate. The reset gate r_t controls how much prior hidden state informs the candidate, enabling the model to ignore irrelevant history.

Two GRU layers are stacked so that the second layer can form higher-order abstractions over the patterns extracted by the first. The final hidden state h_M from the top layer is fed into a fully connected linear layer to produce the scalar prediction:

$$\hat{y} = W_o h_M + b_o$$

4.2 Hyperparameter Selection

- **Hidden size:** 64 (uniform across all stocks)
- **GRU layers:** 2
- **Dropout:** 0.0
- **Epochs:** 50
- **Learning rate:** 0.001 (Adam optimizer)

- **Batch size:** 50
- **Loss function:** Mean Squared Error (MSE)

The hidden size of 64 was chosen to give the two-layer GRU sufficient representational capacity without introducing excessive parameters that could lead to overfitting on the roughly 700 training windows available per stock. Adam was retained from the baseline for its adaptive learning rate properties. Fifty epochs were sufficient for the GRU to converge, as confirmed by the flattening of training loss curves in Figure 2. Mini-batches of 50 balance gradient noise and training speed.

4.3 Loss Curves

Figure 2 shows the MSE training loss over 50 epochs for each stock. All four curves descend sharply in the first 10–15 epochs and stabilize, indicating successful convergence with minimal oscillation.

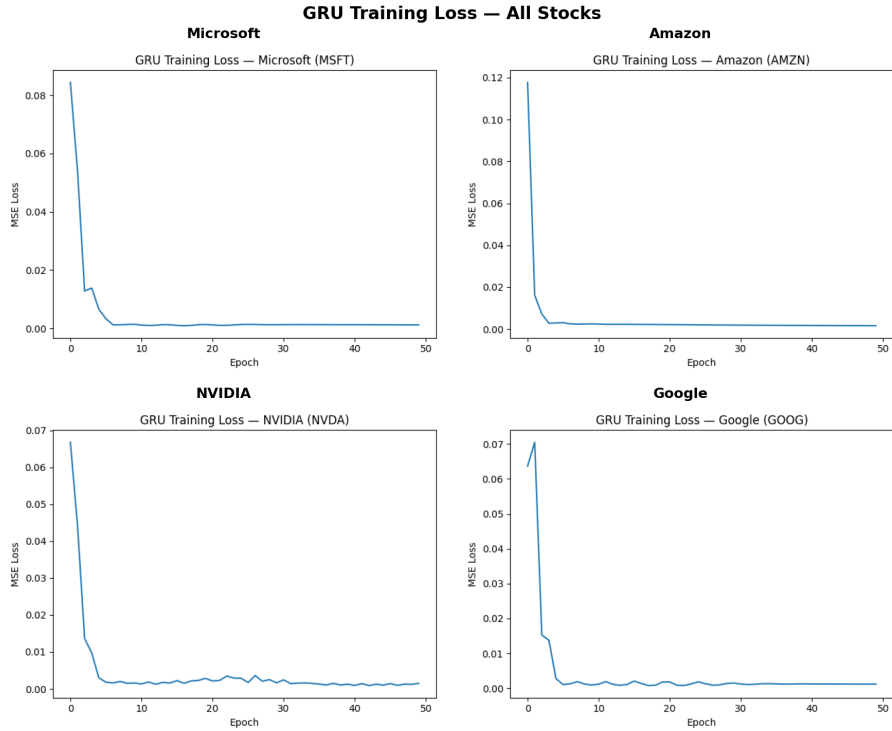


Figure 2: GRU training loss (MSE) over 50 epochs for all four stocks.

4.4 Results

Table 2 summarizes the training time, training error, and test error for each stock. Error is reported as MAPE; accuracy as $100 - \text{MAPE}$.

Stock	Train Time (s)	Train MAPE (%)	Test MAPE (%)	Test Accuracy (%)
Microsoft	17.75	1.71	1.43	98.57
Amazon	17.37	2.42	1.93	98.07
NVIDIA	17.58	6.06	3.00	97.00
Google	17.48	1.67	1.41	98.59

Table 2: GRU results per stock.

All four stocks achieve test accuracies above 95%. Compared to the baseline RNN (Table 1), the GRU improves test MAPE on every stock: Microsoft improves from 1.84% to 1.43%, Amazon from 1.98% to 1.93%, NVIDIA from 4.01% to 3.00%, and Google from 1.45% to 1.41%. The most significant gain is on NVIDIA, where the GRU’s gating mechanism better handles the higher price volatility over the test period. The training time roughly doubles relative to the RNN (17–18 s vs. 8–9 s for the same number of epochs) due to the additional gating computations, but this cost is justified by the accuracy improvement, particularly for NVIDIA.

4.5 Predictions

Figure 3 overlays the GRU predictions against the true closing prices on the test set for each stock.

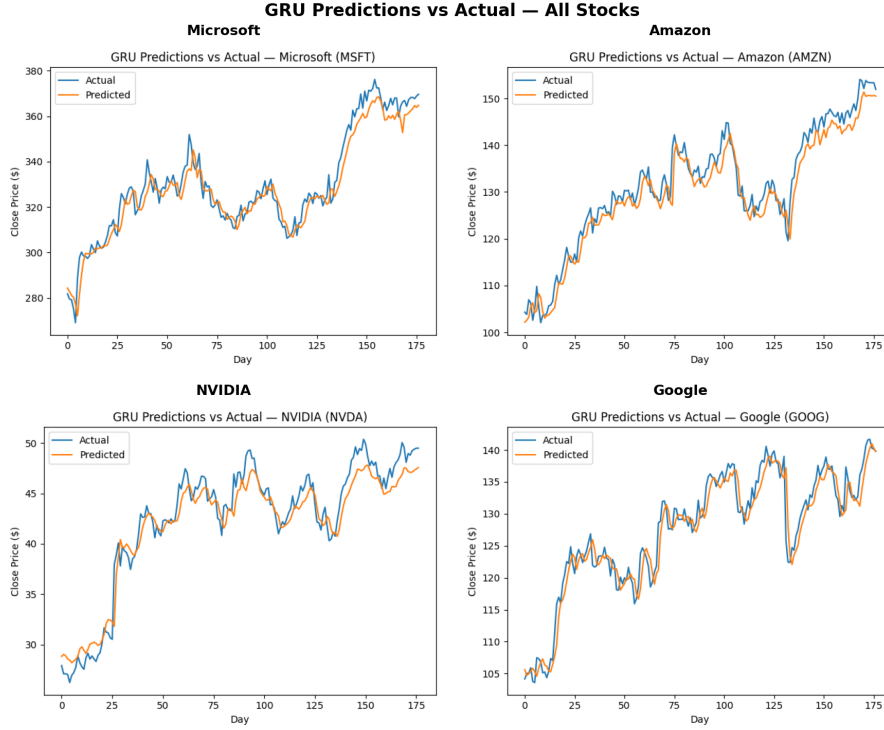


Figure 3: GRU predicted vs. actual closing prices on the test set for all four stocks.

5 Task 5

For Task 5, we implemented a Long Short-Term Memory (LSTM) model to predict the next day’s closing stock price using the previous 60 days of stock data. Each input sequence consisted of five features: closing price, high price, low price, opening price, and volume. The model architecture includes an LSTM recurrent layer followed by a fully connected linear output layer. LSTMs are designed to capture long-term dependencies in sequential data, making them well-suited for time series tasks such as stock prediction.

The model was trained separately on four stocks: Microsoft, Google, Amazon, and NVIDIA. For each stock, the dataset produced 886 total windows, with 709 used for training and 177 for testing, maintaining an approximate 80%/20% split. The model was trained using Mean Squared Error (MSE) loss and the Adam optimizer. Early stopping was applied based on validation loss to prevent overfitting.

During training, the loss decreased rapidly in early epochs and then stabilized as the model converged. For example, Microsoft’s training loss decreased from approximately 0.17 in the first epoch to around 0.001–0.002 near con-

vergence. Similar behavior was observed for Google and Amazon. NVIDIA exhibited more unstable validation loss, likely due to higher volatility in its stock price, making it more difficult to model accurately.

5.1 Results

Stock	Train MAPE	Train Acc.	Test MAPE	Test Acc.	Time (s)
Microsoft	2.3742%	97.63%	1.7459%	98.25%	9.90
Google	2.0774%	97.92%	1.7228%	98.28%	22.72
Amazon	2.0855%	97.91%	1.5848%	98.42%	25.90
NVIDIA	9.7145%	90.29%	3.9201%	96.08%	3.11

Table 3: LSTM model performance across four stocks

All four stocks achieved a test accuracy above the required 95%. Amazon achieved the best performance with the lowest test MAPE of 1.5848%, while NVIDIA had the highest error with a test MAPE of 3.9201%. Despite this, NVIDIA still satisfied the accuracy requirement.

The training and test errors for Microsoft, Google, and Amazon are close in value, indicating that the model generalizes well without significant overfitting. NVIDIA showed greater variability in performance, which is likely due to higher volatility and more complex patterns in its price data.

Overall, the LSTM model outperformed the baseline RNN by effectively capturing long-term dependencies in the time series. This demonstrates that LSTM is well-suited for stock prediction.

5.2 Visualization

To further evaluate performance, we plotted the true closing prices against the predicted closing prices for each stock. These figures show that the model is able to closely follow the overall trend of stock price movement, although short-term fluctuations are sometimes missed.

